

A PROJECT REPORT ON

FAKE NEWS AND MISINFORMATION DETECTOR FOR TAMIL AND ENGLISH

Domain: NLP / Social Media

Submitted in partial fulfilment of the requirements for the award of the degree of

B.Sc. / B.E. / MCA – COMPUTER SCIENCE

Submitted by

STUDENT NAME

(Register No: XXXXXXXXX)

Under the guidance of

GUIDE NAME, M.Sc., M.Phil.,

Assistant Professor

DEPARTMENT OF COMPUTER SCIENCE

COLLEGE NAME

2025 – 2026

CONTENTS

CHAPTER	TITLE	PAGE
I	INTRODUCTION	1
	1.1 History of Domain	
	1.2 Overview of the Project	
	1.3 Motivation	
	1.4 Aim of the Project	
	1.5 Objectives	
	1.6 Literature Review	
II	SOFTWARE PROJECT PLAN	5
	2.1 Existing System	
	2.2 Proposed System	
	2.3 Project Schedule	
III	CUSTOMER REQUIREMENT DETERMINATION	7
	3.1 Functional Requirements	
	3.2 Non-functional Requirements	
	3.3 User Requirements	
	3.4 System Requirements	
IV	SOFTWARE REQUIREMENT SPECIFICATION	9
	4.1 Hardware Specification	
	4.2 Software Specification	
V	SYSTEM ANALYSIS	10
	5.1 Feasibility Study	

	5.2 About the Software	
VI	SYSTEM DESIGN	14
	6.1 UML Diagram	
	6.2 System Architecture	
	6.3 Data Flow Diagram	
	6.4 ER Diagram	
	6.5 Table Design	
	6.6 Algorithm Design	
	6.7 Modules	
VII	IMPLEMENTATION AND TESTING	23
	7.1 Introduction	
	7.2 Implementation	
	7.3 Test Cases	
	7.4 Source Code	
	7.5 Screen Layout	
VIII	CONCLUSION AND FUTURE ENHANCEMENT	31
	8.1 Conclusion	
	8.2 Future Enhancement	
IX	BIBLIOGRAPHY	32

CHAPTER I

INTRODUCTION

Social media platforms and messaging applications such as WhatsApp, Facebook, X (Twitter) and YouTube have become the main source of news for millions of people. While they make information sharing fast and easy, they also allow false and misleading content to spread to thousands of people within minutes. Forwarded messages about miracle cures, fake government schemes, communal rumours and edited election claims are common examples.

Misinformation spreads faster than corrections. Studies of Twitter data have shown that false stories reach people more quickly and more widely than true ones. In India the problem is made worse because a large amount of content is circulated in regional languages such as Tamil, while most automatic fact-checking tools work only in English.

Natural Language Processing (NLP) and transformer-based language models such as BERT can learn the linguistic patterns that are typical of fake news, such as sensational words, emotional tone and lack of sources. This project, TruthLens, is a web application and browser extension that checks whether a news headline or WhatsApp-style message is likely fake, cross-checks the claim with trusted news sources and fact-check databases, and explains why the message was flagged.

LIMITATIONS

1. Manual fact-checking is slow and cannot keep up with the volume of viral messages.
2. Most automatic detectors support only English.
3. Users receive only a label without any explanation, so they do not trust the result.
4. Fact-check articles are scattered across many websites.

1.1 History of Domain

Propaganda and false rumours are as old as human communication, but the term 'fake news' became widely used during the 2016 United States presidential election.

Dedicated fact-checking organisations such as PolitiFact (2007) and Snopes (1994) were followed in India by Alt News (2017), BOOM and Factly, and Google introduced the ClaimReview markup so that fact-check articles could be discovered by search engines.

Early automatic detection used hand-crafted linguistic features with classifiers such as Naive Bayes and SVM on datasets like LIAR (2017). The introduction of BERT in 2018 and its multilingual versions (mBERT, XLM-RoBERTa, IndicBERT, MuRIL) allowed models to understand context in many Indian languages, enabling misinformation detection in Tamil.

1.2 Overview of the Project

The user pastes text or a URL into the TruthLens web page or right-clicks a selected message in the browser extension. The system cleans the text, detects whether it is Tamil, English or mixed (Tanglish), and generates contextual embeddings using a fine-tuned multilingual BERT model (XLM-RoBERTa). A classification head predicts whether the content is Real, Fake or Uncertain.

In parallel, keywords are extracted and used to search trusted news APIs and the Google Fact Check Tools API. The semantic similarity between the message and retrieved articles is computed. A final score combines the model probability and the source-match score. The verdict is shown with highlighted suspicious phrases (using attention/LIME weights) and links to matching fact-check articles. Users can give feedback that is used to retrain the model.

1.3 Motivation

Ordinary users, especially elderly people and first-time internet users, find it hard to verify viral messages, and wrong information can cause panic, financial loss or even violence.

- **Motivation:** To give every user a simple tool that checks a message in their own language and explains the result.
- **Outcome:** Reduced spread of rumours, increased awareness, and a feedback-driven Tamil misinformation dataset.

1.4 Aim of the Project

The project is entitled as “**FAKE NEWS AND MISINFORMATION DETECTOR FOR TAMIL AND ENGLISH**”. The aim of this project is to develop a bilingual (Tamil and English) misinformation detection system that classifies news text using transformer models, verifies claims against trusted sources and presents an explainable verdict through a web application and a browser extension.

1.5 Objectives

- To collect and label a Tamil-English fake news dataset.
- To fine-tune a multilingual transformer for real/fake/uncertain classification.
- To cross-check claims with trusted news and fact-check APIs.
- To highlight suspicious phrases to explain the decision.
- To collect user feedback for continuous improvement.

1.6 Literature Review

Shu et al. (2017): Presented a comprehensive survey of fake news detection on social media from a data mining perspective, categorising methods into news content-based and social context-based approaches.

Wang (2017): Released the LIAR dataset of 12.8K manually labelled short statements from PolitiFact with six truthfulness labels, and showed that combining metadata with text improves detection.

Vosoughi, Roy and Aral (2018): Analysed about 126,000 rumour cascades on Twitter and found that false news spread farther, faster, deeper and more broadly than the truth in all categories of information.

Devlin et al. (2019): Introduced BERT, a deeply bidirectional transformer pre-trained on masked language modelling, which achieved state-of-the-art results on many NLP tasks after fine-tuning.

Conneau et al. (2020): Proposed XLM-RoBERTa, a cross-lingual model trained on 2.5 TB of CommonCrawl data in 100 languages including Tamil, which performs well on low-resource languages.

Ribeiro, Singh and Guestrin (2016): Proposed LIME, a technique that explains the predictions of any classifier by learning an interpretable model locally around the prediction.

CHAPTER II

SOFTWARE PROJECT PLAN

2.1 Existing System

Currently users depend on manual fact-checking websites, news channels or friends to verify viral messages. Some platforms add labels to a few posts. Research models exist mostly for English datasets and are not available as easy tools for common users. They return only a label without reasons or sources.

DISADVANTAGES

- Manual verification is slow; rumours spread before correction.
- No support for Tamil and code-mixed text.
- No explanation of why the content is fake.
- Users must visit multiple websites to verify.
- No feedback mechanism for improvement.

2.2 Proposed System

The proposed TruthLens system combines a fine-tuned multilingual transformer classifier with evidence retrieval from trusted sources. It supports Tamil, English and code-mixed text, gives a three-way verdict (Real, Fake, Uncertain), highlights suspicious phrases, lists supporting or contradicting articles, and learns from user feedback. It is available as a web application and a Chrome extension.

ADVANTAGES

- Works for Tamil, English and mixed text.
- Evidence-based verdict with source links.
- Explainable output with highlighted phrases.
- One-click checking through browser extension.
- Continuously improves with feedback.

2.3 Project Schedule

The development of TruthLens follows the iterative waterfall model. Each phase produces a deliverable that is reviewed before the next phase begins, and feedback from testing is fed back into design and implementation. The planned schedule is shown below.

S.No	Phase	Duration
1	Requirement analysis and problem study	Week 1 - 2
2	Literature survey and existing system study	Week 3 - 4
3	Dataset collection / hardware procurement	Week 5 - 6
4	System design (DFD, UML, ER, table design)	Week 7 - 8
5	Implementation of modules	Week 9 - 12
6	Testing and model evaluation	Week 13 - 14
7	Documentation and final review	Week 15 - 16

CHAPTER III

CUSTOMER REQUIREMENT DETERMINATION

3.1 Functional Requirements

Functional requirements define the specific features and operations a system must perform to meet business and user needs. They describe what the system should do and how it interacts with users or other systems. They focus on the behaviour and functionality of the system and represent features that can be directly observed and tested in the final product. The functional requirements of the TruthLens system are listed below.

- **Input acceptance:** Accept plain text, a URL or an image caption.
- **Language detection:** Identify Tamil, English or code-mixed text.
- **Classification:** Predict Real, Fake or Uncertain with probability.
- **Evidence search:** Query trusted news and fact-check APIs and compute similarity.
- **Score fusion:** Combine model score and source-match score into final verdict.
- **Explanation:** Highlight suspicious phrases and show reasons.
- **Feedback:** Allow users to mark the verdict as correct or wrong.
- **Admin:** Manage trusted source list and retrain model.

3.2 Non-functional Requirements

Non-functional requirements determine the performance standards and quality attributes of software, e.g. system usability, effectiveness, security, scalability, etc. While functional requirements determine what the system does, non-functional requirements describe HOW the system does it. If an application does not meet non-functional requirements, it continues to perform its basic functions; however, it will not be able to provide a good user experience. Therefore, non-functional requirements are as important as functional requirements for successful product adoption.

- **Performance:** Verdict within 4 seconds for a 100-word message.
- **Accuracy:** Macro F1-score of at least 0.85 on the test set.

- **Usability:** Bilingual interface, one-click check.
- **Privacy:** Checked messages stored without user identity.
- **Scalability:** API can be scaled horizontally.
- **Maintainability:** Model versioning for safe updates.

3.3 User Requirements

- Paste or select any message and check it quickly
- Understand the result in simple Tamil or English
- See trusted links that confirm or deny the claim
- Report wrong results
- Use from the browser without opening another site

3.4 System Requirements

- FastAPI server hosting the transformer model
- PostgreSQL database for checks and feedback
- Access to news and fact-check APIs
- React web application and Chrome extension
- GPU for training (Colab acceptable)

CHAPTER IV

SOFTWARE REQUIREMENT SPECIFICATION

4.1 Hardware Specification

System : Intel i5 Processor 2.4 GHz or above
Hard Disk : 256 GB SSD
RAM : 16 GB
GPU : NVIDIA T4 / Google Colab for training
Monitor : 15" colour monitor

4.2 Software Specification

Operating System : Windows 10 / Ubuntu 22.04
Coding Language : Python 3.10, JavaScript
NLP Library : HuggingFace Transformers, PyTorch
Model : XLM-RoBERTa base (fine-tuned)
Back End : FastAPI
Front End : React JS, Chrome Extension API
Database : PostgreSQL
External APIs : Google Fact Check Tools API, NewsAPI

CHAPTER V

SYSTEM ANALYSIS

5.1 Feasibility Study

A feasibility study is carried out to find whether the proposed TruthLens system is worth developing. Three key considerations are involved in the feasibility analysis: technical, economic and operational feasibility.

TECHNICAL FEASIBILITY

Pre-trained multilingual transformers are freely available through HuggingFace and can be fine-tuned on a few thousand labelled samples using a free Colab GPU. FastAPI and React are mature technologies. The project is technically feasible.

ECONOMIC FEASIBILITY

All libraries are open source and the fact-check API is free for limited use. The model can be served on a CPU server for a small number of users. The system is economically feasible.

OPERATIONAL FEASIBILITY

Users only need to paste a message and click Check. The extension works from the right-click menu. The verdict is in simple language with colours, so it is operationally feasible.

5.2 About the Software

PYTHON

Python is a high-level, interpreted, general-purpose programming language created by Guido van Rossum and first released in 1991. Its design philosophy emphasises code readability through significant indentation, and its simple syntax allows developers to express concepts in fewer lines of code than languages such as C++ or Java.

Python supports multiple programming paradigms including procedural, object-oriented and functional programming. It has a very large standard library and an ecosystem of third-party packages available through the Python Package Index (PyPI), which makes it the most popular language for data science, machine learning, automation and back-end web development.

- Easy to learn, read and maintain
- Huge library support for AI, ML and data processing (NumPy, Pandas, scikit-learn)
- Platform independent - runs on Windows, Linux and macOS
- Strong community support and documentation

HUGGINGFACE TRANSFORMERS

HuggingFace Transformers is an open-source library that provides thousands of pre-trained transformer models for tasks such as text classification, named entity recognition, question answering and translation, in more than 100 languages.

Multilingual models such as mBERT, XLM-RoBERTa and IndicBERT understand Tamil and English, so a single model can be fine-tuned to classify text in both languages.

- Pre-trained multilingual models
- Simple pipeline() API
- Works with both PyTorch and TensorFlow

PYTORCH

PyTorch is an open-source deep learning framework originally developed by Meta AI. It uses dynamic computation graphs, which means the network is built at run time; this makes debugging simple and the code feels like normal Python.

PyTorch is widely used for research and production in computer vision and natural language processing. The torch.nn module provides layers such as LSTM, GRU and convolution, and models can be exported using TorchScript or ONNX for deployment.

- Dynamic graphs and pythonic debugging

- Rich ecosystem (TorchVision, TorchText, HuggingFace)
- GPU acceleration through CUDA

SENTENCE-TRANSFORMERS (SBERT)

Sentence-Transformers is a Python framework for state-of-the-art sentence, paragraph and document embeddings. It modifies BERT using siamese and triplet network structures so that semantically similar sentences are placed close together in vector space.

Similarity between two texts is calculated using cosine similarity of their embeddings, which captures meaning rather than exact words; for example 'ML engineer' and 'machine learning developer' get a high score.

FASTAPI

FastAPI is a modern, high-performance web framework for building APIs with Python based on standard Python type hints. It is built on Starlette for the web parts and Pydantic for data validation.

FastAPI automatically generates interactive API documentation (Swagger UI and ReDoc), validates request bodies, and supports asynchronous request handling, which gives performance comparable to Node.js and Go. It is well suited for serving machine learning models with low latency.

- Automatic request validation with Pydantic
- Auto-generated OpenAPI documentation
- Asynchronous and very fast

REACT JS

React is an open-source JavaScript library for building user interfaces, maintained by Meta and a community of developers. It lets developers build reusable UI components and manages the view layer of web applications efficiently using a virtual DOM.

React applications are composed of components that hold their own state and re-render automatically when data changes. With libraries such as React Router and Axios, a

single-page application can communicate with REST APIs and give the user a fast, app-like experience.

- Component based, reusable UI
- Virtual DOM for fast rendering
- Large ecosystem of UI libraries

CHROME EXTENSION API (MANIFEST V3)

Chrome extensions are small software programs that customise the browsing experience. They are built with HTML, CSS and JavaScript and use the chrome.* APIs such as webNavigation, tabs, storage and declarativeNetRequest.

Manifest V3 uses a background service worker that can observe every navigation and communicate with a content script injected into the page to show warnings.

POSTGRESQL

PostgreSQL is a powerful, open-source object-relational database system with over 35 years of active development. It has earned a strong reputation for reliability, feature robustness and performance.

PostgreSQL supports ACID transactions, foreign keys, views, triggers, stored procedures and JSON data types. Its extension system allows additional capabilities such as PostGIS for geographic data and pgvector for similarity search.

- Fully ACID compliant
- Advanced SQL and JSON support
- Extensible with PostGIS and other extensions

CHAPTER VI

SYSTEM DESIGN

System design is the process of defining the architecture, components, modules, interfaces and data of a system to satisfy the specified requirements. The design of TruthLens is presented using UML use case diagram, system architecture, data flow diagrams, ER diagram and table design.

6.1 UML Diagram

USE CASE DIAGRAM

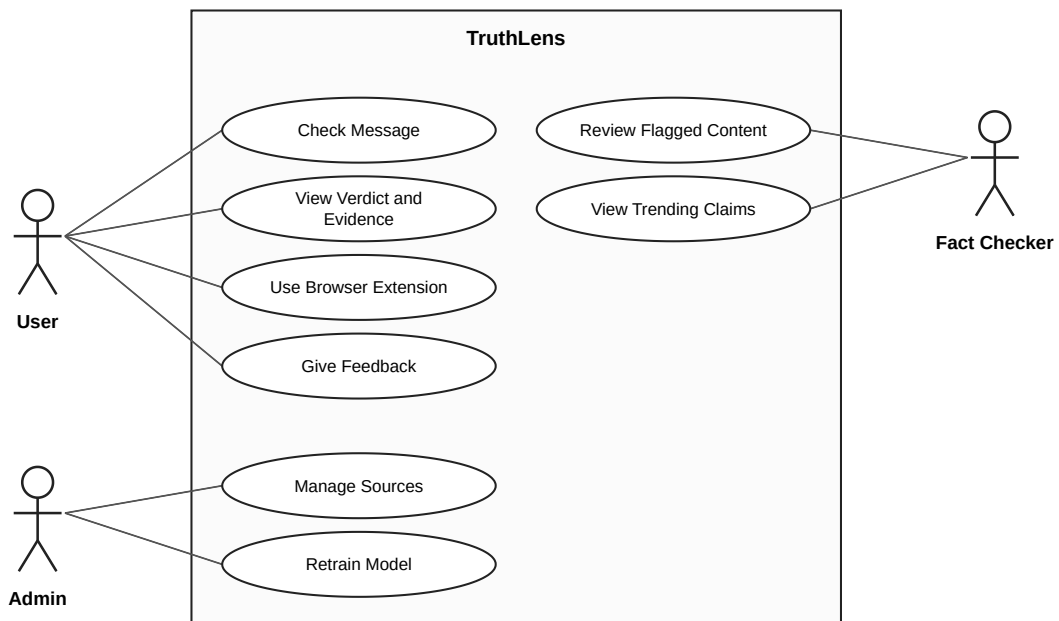


Fig 1: Use case diagram of TruthLens

6.2 System Architecture

TruthLens uses a client-server architecture. The web app and the Chrome extension act as clients. The FastAPI server hosts the language detector, the transformer classifier and the evidence retrieval module. PostgreSQL stores checks, feedback and source lists.

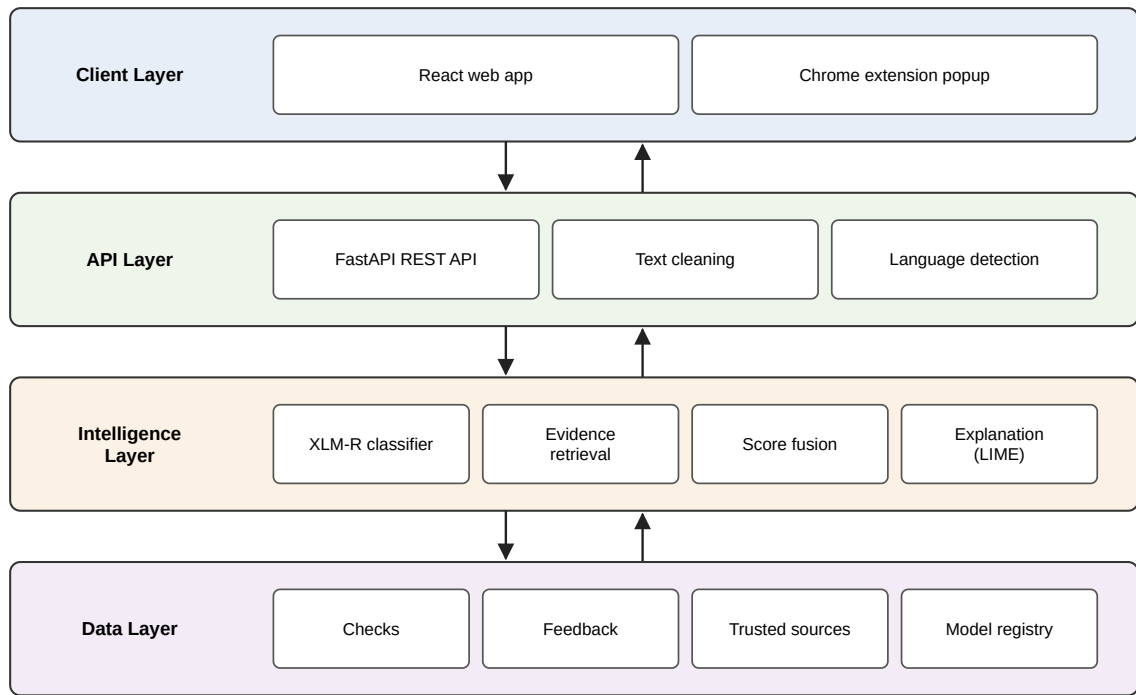


Fig 2: System architecture

6.3 Data Flow Diagram

A Data Flow Diagram (DFD) is a graphical representation of the flow of data through an information system. The Level 0 DFD (context diagram) shows the whole system as a single process with its external entities, and the Level 1 DFD breaks it into its major processes and data stores.

LEVEL 0 DFD (CONTEXT DIAGRAM)

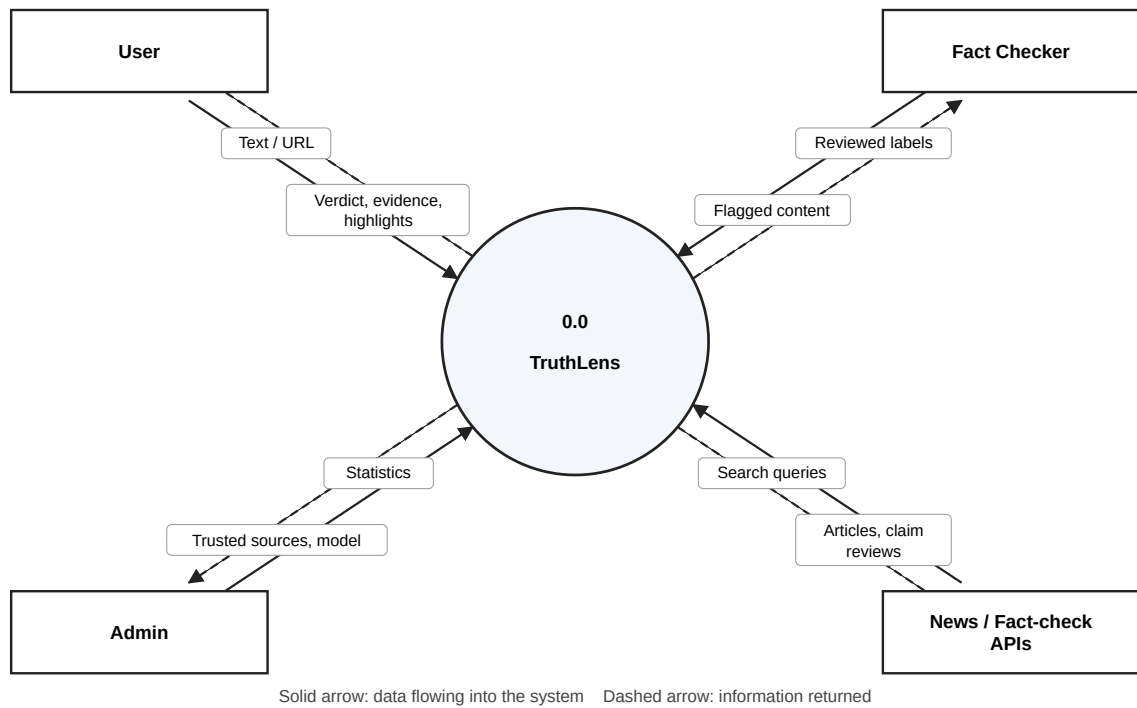


Fig 3: Level 0 data flow diagram

LEVEL 1 DFD

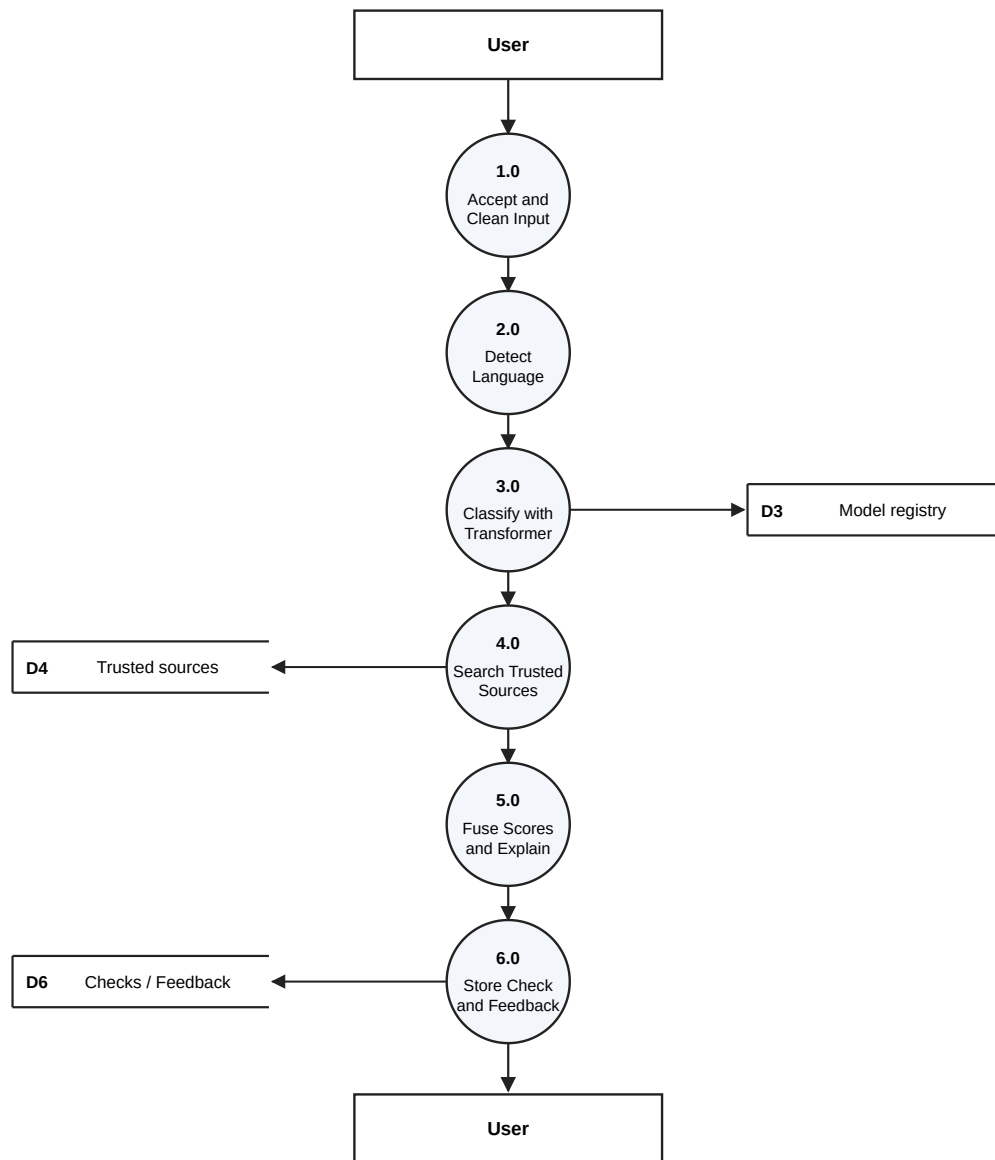


Fig 4: Level 1 data flow diagram

6.4 ER Diagram

The Entity Relationship (ER) diagram shows the entities stored in the database, their key attributes and the relationships between them. 1:N indicates a one-to-many relationship.

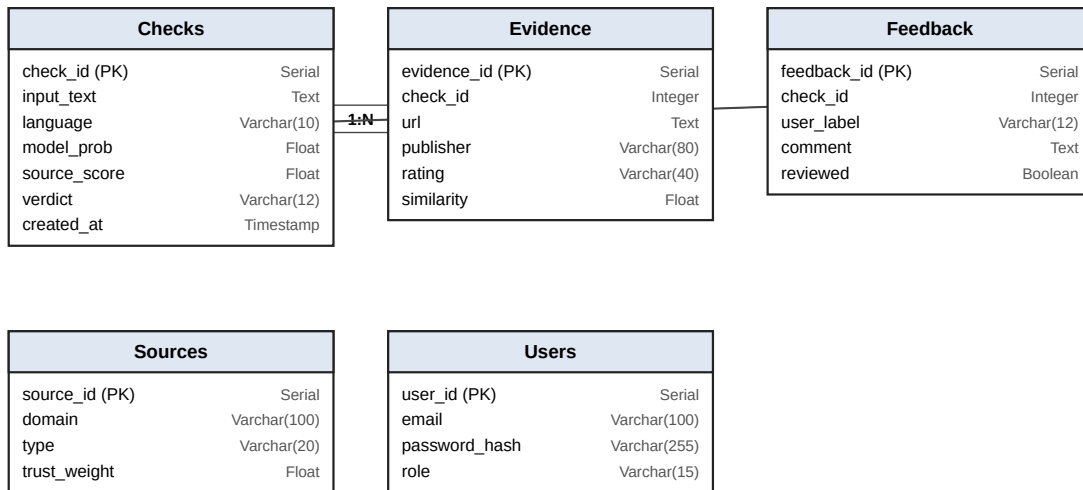


Fig 5: Entity relationship diagram

6.5 Table Design

CHECKS

FIELD NAME	DATA TYPE	DESCRIPTION
check_id	Serial	Primary key
input_text	Text	Message checked
language	Varchar(10)	ta / en / mixed
model_prob	Float	Fake probability
source_score	Float	Evidence match score
verdict	Varchar(12)	real / fake / uncertain
created_at	Timestamp	Time

EVIDENCE

FIELD NAME	DATA TYPE	DESCRIPTION
evidence_id	Serial	Primary key
check_id	Integer	Foreign key to Checks
url	Text	Article URL
publisher	Varchar(80)	Source name
rating	Varchar(40)	Fact-check rating
similarity	Float	Cosine similarity

FEEDBACK

FIELD NAME	DATA TYPE	DESCRIPTION
feedback_id	Serial	Primary key
check_id	Integer	Foreign key to Checks
user_label	Varchar(12)	Label given by user
comment	Text	Optional comment
reviewed	Boolean	Reviewed by fact checker

SOURCES

FIELD NAME	DATA TYPE	DESCRIPTION
source_id	Serial	Primary key
domain	Varchar(100)	Trusted domain
type	Varchar(20)	news / factcheck
trust_weight	Float	Weight in scoring

USERS

FIELD NAME	DATA TYPE	DESCRIPTION
user_id	Serial	Primary key
email	Varchar(100)	Login e-mail
password_hash	Varchar(255)	Hashed password
role	Varchar(15)	checker / admin

6.6 Algorithm Design

The detection algorithm fuses the probability from a fine-tuned XLM-RoBERTa classifier with an evidence score obtained from trusted sources.

1. **Accept text or URL input** – If a URL is given, the article title and body are extracted.
2. **Clean text and detect language** – URLs, emojis and repeated characters are removed; the langdetect library and Tamil Unicode range check identify the

language.

3. **Generate embeddings with a multilingual BERT model** – Text is tokenised with the XLM-R tokenizer (max 256 tokens) and passed through the encoder.
4. **Classify as real/fake/uncertain** – A softmax head outputs P(real) and P(fake); if the maximum is below 0.65 the text is Uncertain.
5. **Search trusted sources via news APIs and compare similarity** – Keywords are sent to the Fact Check API and NewsAPI; SBERT cosine similarity is computed for each returned headline.
6. **Combine model score and source-match score** – Final fake score = $0.6 \times \text{model score} + 0.4 \times \text{evidence score}$, where a matching 'False' rating raises and a matching trusted news report lowers the score.
7. **Show verdict with highlighted suspicious phrases** – LIME weights identify words that contributed most to the fake prediction.
8. **Collect user feedback to retrain** – Feedback reviewed by fact checkers is added to the training set.

PSEUDO CODE

```
FUNCTION CHECK(input)
  text <- CLEAN(EXTRACT(input)); lang <- DETECT_LANGUAGE(text)
  p_fake <- SOFTMAX(XLMR(TOKENIZE(text)))[fake]
  claims <- FACTCHECK_API(KEYWORDS(text)) + NEWS_API(KEYWORDS(text))
  ev <- 0.5
  FOR c IN claims WITH SIM(text, c.title) > 0.7:
    IF c.rating IN {False, Misleading} THEN ev <- max(ev, 0.9)
    ELSE IF c.source IS trusted news THEN ev <- min(ev, 0.1)
  score <- 0.6 * p_fake + 0.4 * ev
  verdict <- FAKE if score > 0.65, REAL if score < 0.35, else UNCERTAIN
  RETURN verdict, score, LIME_HIGHLIGHT(text), claims
```

6.7 Modules

The application is divided into modules for the general User, the volunteer Fact Checker and the Admin.

MODULES

USER

1. CHECK MESSAGE
2. VIEW VERDICT AND EVIDENCE
3. USE BROWSER EXTENSION
4. GIVE FEEDBACK

FACT CHECKER

1. REVIEW FLAGGED CONTENT
2. VIEW TRENDING CLAIMS

ADMIN

1. MANAGE SOURCES
2. RETRAIN MODEL

USER

CHECK MESSAGE

The user pastes a message, headline or URL and clicks Check. For URLs the article text is extracted using a readability parser before analysis.

VIEW VERDICT AND EVIDENCE

The verdict (Real, Fake, Uncertain), confidence, highlighted phrases and matching fact-check articles are displayed.

USE BROWSER EXTENSION

The user selects text on any web page, right-clicks and chooses 'Check with TruthLens'; the result appears in a popup.

GIVE FEEDBACK

The user marks the verdict as correct or wrong and optionally adds a source link.

FACT CHECKER

REVIEW FLAGGED CONTENT

Volunteer fact checkers review uncertain and disputed messages and assign the correct label.

VIEW TRENDING CLAIMS

Shows the most frequently checked claims in the last 24 hours.

ADMIN

MANAGE SOURCES

Admin maintains the list of trusted news domains and fact-check sites.

RETRAIN MODEL

Admin exports reviewed feedback as training data and deploys a new model version.

CHAPTER VII

IMPLEMENTATION AND TESTING

7.1 Introduction

TruthLens was implemented with a Python FastAPI back end and a React front end. The classifier was fine-tuned in PyTorch using the HuggingFace Trainer API on Google Colab. The Chrome extension was built with Manifest V3 and communicates with the same API.

7.2 Implementation

The training data combined the English LIAR and FakeNewsNet samples with around 4,000 Tamil and code-mixed messages collected from public fact-check articles (labelled fake) and mainstream Tamil news headlines (labelled real). The data was split 80:10:10. Fine-tuning was done for 4 epochs with a learning rate of $2e-5$ and batch size 16.

Testing is a process of running software with the intention of finding errors. The software which has been developed has to be tested to prove its validity. Testing is considered to be the least creative phase of the whole cycle of system design; in the real sense it is the phase which helps to bring out the creativity of the other phases. Testing a program is the first step in the debugging process.

A good testing suite is vital because all possible execution paths inside the code must be tested in order to have any hope that it is bug free. Testing of the system is done to ensure the integrity of the system. The aim of testing is to prove that the developed system addresses the predefined processing requirements and will perform reliably and efficiently during operation.

PURPOSES OF TESTING

- To affirm the quality of the project.
- To find and eliminate any errors from previous stages.
- To validate the software against its requirements.

- To ensure the reliability of the system.

UNIT TESTING

In the lines of this strategy all the individual functions and modules were put to the test independently. By following this strategy all the errors in coding were identified and corrected. This method was applied in combination with white box and black box testing techniques to find the errors in each module.

INTEGRATION TESTING

This testing strategy has two different approaches, namely the top-down approach, in which integration is carried out from the top-level module to the bottom, and the bottom-up approach, in which integration is carried out from the low-level modules to the top. The modules were tested using the bottom-up approach by introducing stubs for the top-level functions. This test was used to identify errors in the interfaces and in passing parameters between functions.

VALIDATION TESTING

The main aim of this testing is to verify that the software system does what it was designed for. Alpha testing and beta testing were conducted to ensure the validity of the system. Alpha testing was done at the developer's site and beta testing was carried out by sample end users.

SYSTEM TESTING

The idea of applying this strategy was to put the system through a series of different tests whose primary purpose is to fully exercise the computer-based system. Although each test has a different purpose, all work to verify that all system elements have been properly integrated and perform their allocated functions. Security testing and performance testing were also conducted.

ACCEPTANCE TESTING

Acceptance testing involves planning and execution of functional tests, performance tests and stress tests to verify that the implemented system satisfies its

requirements. Typically, the acceptance test incorporates test cases developed during unit testing and integration testing.

7.3 Test Cases

Test ID	Test Description	Input	Expected Output	Result
TC01	English fake message	'Govt giving free laptop to all, click link'	Verdict FAKE with fact-check link	Pass
TC02	Tamil real headline	Official weather department news in Tamil	Verdict REAL	Pass
TC03	Code-mixed text	Tanglish rumour message	Language = mixed, verdict FAKE	Pass
TC04	URL input	News article URL	Article text extracted and checked	Pass
TC05	Empty input	Blank text	Error 'Please enter a message'	Pass
TC06	Highlights	Sensational message	Words like 'urgent', 'share' highlighted	Pass
TC07	Extension check	Select text, right click	Popup with verdict	Pass
TC08	Feedback	Mark verdict as wrong	Feedback saved for review	Pass

PERFORMANCE EVALUATION

The model was evaluated on the test set of 1,050 messages.

Model	Accuracy	Macro F1	Tamil F1	English F1
TF-IDF + SVM	78.2%	0.76	0.69	0.81
mBERT	86.5%	0.85	0.82	0.88
XLM-R base (proposed)	89.7%	0.89	0.87	0.90
XLM-R + evidence fusion	92.1%	0.91	0.90	0.92

7.4 Source Code

train_classifier.py

```
import pandas as pd, numpy as np
from datasets import Dataset
from transformers import (AutoTokenizer, AutoModelForSequenceClassification,
                          TrainingArguments, Trainer)
from sklearn.metrics import f1_score, accuracy_score

MODEL = "xlm-roberta-base"
df = pd.read_csv("data/truthlens_dataset.csv")          # columns: text, label
(0 real, 1 fake)
train_df = df.sample(frac=0.8, random_state=42); rest = df.drop(train_df.index)
val_df = rest.sample(frac=0.5, random_state=42); test_df =
rest.drop(val_df.index)

tok = AutoTokenizer.from_pretrained(MODEL)
def encode(b):
    return tok(b["text"], truncation=True, padding="max_length", max_length=256)

train_ds = Dataset.from_pandas(train_df).map(encode, batched=True)
val_ds = Dataset.from_pandas(val_df).map(encode, batched=True)

model = AutoModelForSequenceClassification.from_pretrained(MODEL, num_labels=2)

def metrics(p):
    pred = np.argmax(p.predictions, axis=1)
    return {"accuracy": accuracy_score(p.label_ids, pred),
            "f1": f1_score(p.label_ids, pred, average="macro")}

args = TrainingArguments("out", learning_rate=2e-5,
per_device_train_batch_size=16,
                        num_train_epochs=4, evaluation_strategy="epoch",
                        save_strategy="epoch", load_best_model_at_end=True)
trainer = Trainer(model=model, args=args, train_dataset=train_ds,
                  eval_dataset=val_ds, compute_metrics=metrics)
trainer.train()
trainer.save_model("models/truthlens-xlmr");
tok.save_pretrained("models/truthlens-xlmr")
```

api.py

```
import re, os, requests, torch
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from langdetect import detect
from transformers import AutoTokenizer, AutoModelForSequenceClassification
from sentence_transformers import SentenceTransformer, util

app = FastAPI(title="TruthLens API")
tok = AutoTokenizer.from_pretrained("models/truthlens-xlmr")
clf = AutoModelForSequenceClassification.from_pretrained("models/truthlens-xlmr").eval()
sbert = SentenceTransformer("paraphrase-multilingual-MiniLM-L12-v2")
FACT_API = "https://factchecktools.googleapis.com/v1alpha1/claims:search"
BAD = {"false", "misleading", "fake", "pants on fire"}

class Query(BaseModel):
```

```

text: str

def clean(t):
    t = re.sub(r"http\S+", " ", t)
    t = re.sub(r"(\.|\{|\}|\(|\)|\&|\\)", r"\1", t)
    return re.sub(r"\s+", " ", t).strip()

def language(t):
    tamil = sum(1 for ch in t if "\u0B80" <= ch <= "\u0BFF")
    if tamil > 0.6 * len(t.replace(" ", "")): return "ta"
    return "mixed" if tamil else ("en" if detect(t) == "en" else "other")

def model_prob(t):
    enc = tok(t, return_tensors="pt", truncation=True, max_length=256)
    with torch.no_grad():
        return torch.softmax(clf(**enc).logits, -1)[0, 1].item()

def evidence(t):
    r = requests.get(FACT_API, params={"query": t[:120], "key":
os.environ["FC_KEY"]}).json()
    score, found = 0.5, []
    emb = sbert.encode(t, convert_to_tensor=True)
    for c in r.get("claims", []):10]:
        sim = util.cos_sim(emb, sbert.encode(c["text"],
convert_to_tensor=True)).item()
        review = c["claimReview"][0]
        if sim > 0.7:
            found.append({"claim": c["text"], "rating": review["textualRating"],
                        "url": review["url"], "similarity": round(sim, 2)})
            score = 0.9 if review["textualRating"].lower() in BAD else
min(score, 0.2)
    return score, found

@app.post("/check")
def check(q: Query):
    text = clean(q.text)
    if not text:
        raise HTTPException(400, "Please enter a message")
    p = model_prob(text)
    ev, found = evidence(text)
    final = 0.6 * p + 0.4 * ev
    verdict = "FAKE" if final > 0.65 else "REAL" if final < 0.35 else
"UNCERTAIN"
    return {"language": language(text), "model_prob": round(p, 3),
"evidence_score": ev,
        "score": round(final, 3), "verdict": verdict, "evidence": found}

```

extension/background.js

```

chrome.runtime.onInstalled.addListener(() => {
    chrome.contextMenus.create({ id: "truthlens", title: "Check with TruthLens",
contexts: ["selection"] });
});

chrome.contextMenus.onClicked.addListener(async (info, tab) => {
    const res = await fetch("https://api.truthlens.app/check", {
        method: "POST", headers: { "Content-Type": "application/json" },
        body: JSON.stringify({ text: info.selectionText })
    });
    const data = await res.json();
    await chrome.storage.local.set({ last: data });
});

```

```
chrome.action.setBadgeText({ text: data.verdict === "FAKE" ? "!" : "OK",
tabId: tab.id });
chrome.action.setBadgeBackgroundColor({ color: data.verdict === "FAKE" ?
"#d6336c" : "#2f9e44" });
});
```

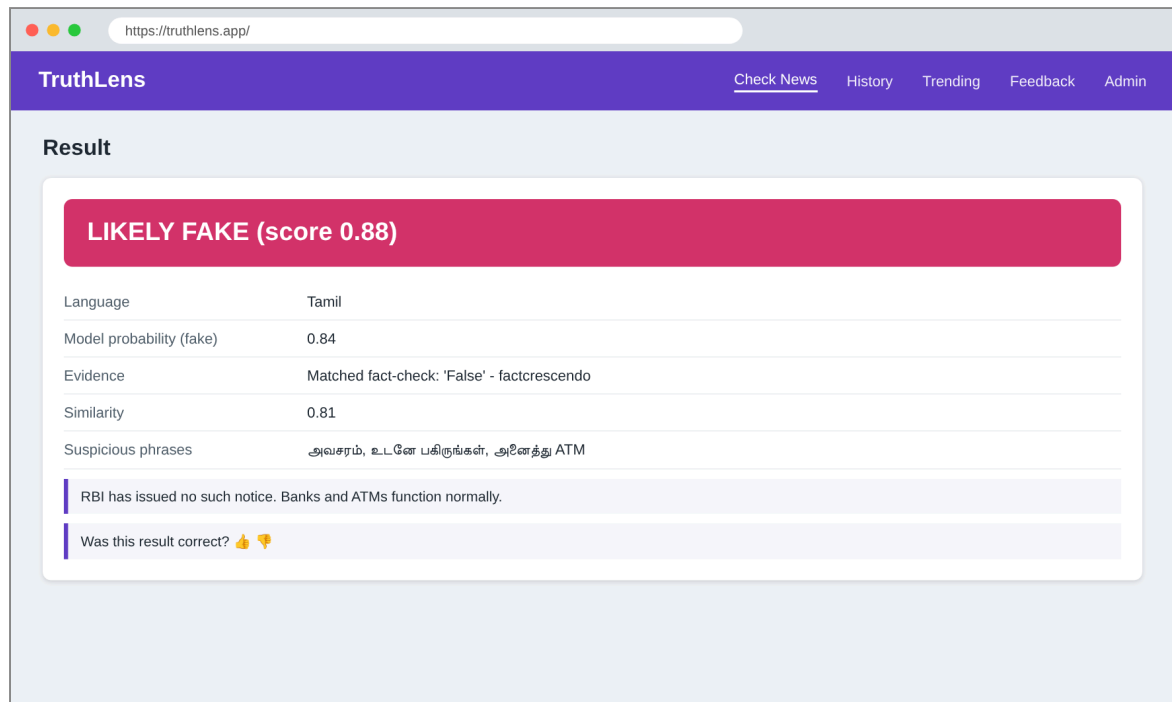
7.5 Screen Layout

SCREENSHOTS:

HOME - CHECK NEWS

The screenshot shows the TruthLens web application interface. The browser's address bar displays `https://truthlens.app/`. The application has a purple header bar with the "TruthLens" logo on the left and navigation links "Check News", "History", "Trending", "Feedback", and "Admin" on the right. The main content area is titled "Check a message or news". It features a form with two input fields: "Paste message / URL" containing the Tamil text "அவசரம்! நாளை முதல் அனைத்து ATM களும் 3 நாட்கள் மூடப்படும். உடனே பகிருங்கள்!" and "Language" set to "Auto detect". A purple "Check Now" button is positioned below the form. To the right of the form is a "Why TruthLens?" section with four bullet points: "Works for Tamil, English and Tenglish.", "Shows trusted sources for every verdict.", "Highlights the words that look suspicious.", and "Your messages are stored without your identity."

VERDICT PAGE



The screenshot shows the TruthLens web application interface. At the top, there's a purple header with the TruthLens logo and navigation links: Check News, History, Trending, Feedback, and Admin. Below the header, the page is titled "Result". A prominent pink banner displays "LIKELY FAKE (score 0.88)". Below this, a table provides details: Language (Tamil), Model probability (fake) (0.84), Evidence (Matched fact-check: 'False' - factcrescendo), Similarity (0.81), and Suspicious phrases (அவசரம், உடனே பகிருங்கள், அனைத்து ATM). A text box contains the statement "RBI has issued no such notice. Banks and ATMs function normally." and a feedback prompt "Was this result correct?" with thumbs up and down icons.

TruthLens

[Check News](#) [History](#) [Trending](#) [Feedback](#) [Admin](#)

Result

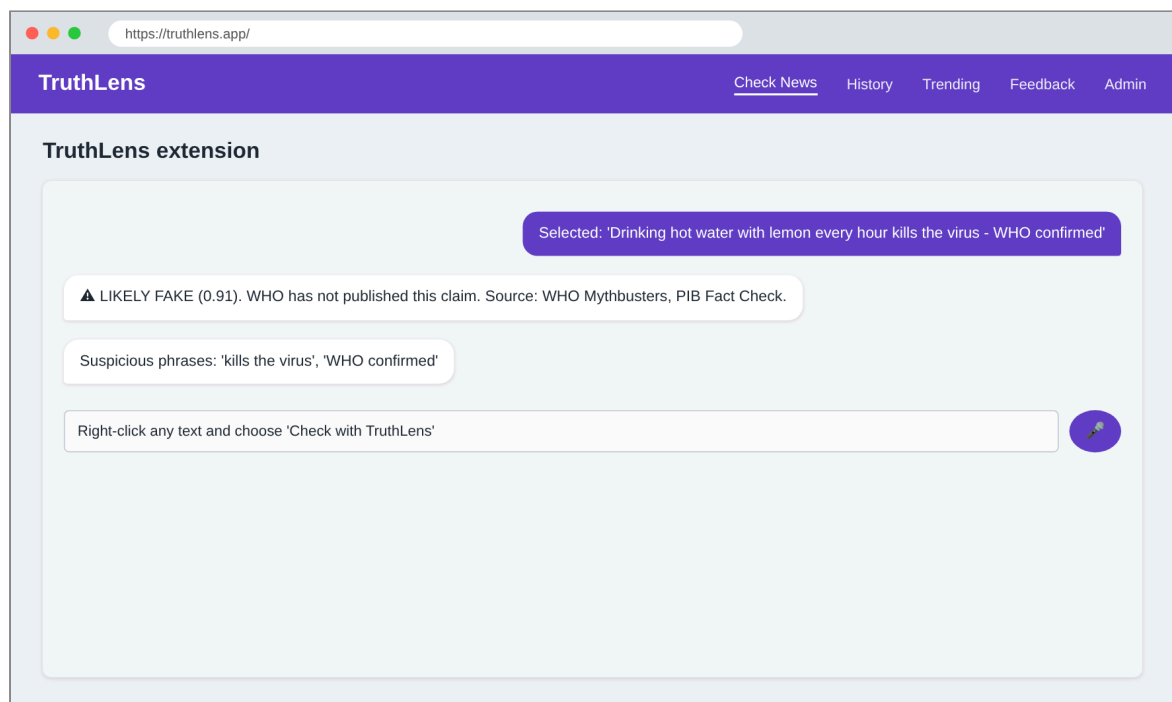
LIKELY FAKE (score 0.88)

Language	Tamil
Model probability (fake)	0.84
Evidence	Matched fact-check: 'False' - factcrescendo
Similarity	0.81
Suspicious phrases	அவசரம், உடனே பகிருங்கள், அனைத்து ATM

RBI has issued no such notice. Banks and ATMs function normally.

Was this result correct? 👍 👎

BROWSER EXTENSION POPUP



The screenshot shows the TruthLens browser extension popup. It features a purple header with the TruthLens logo and navigation links: Check News, History, Trending, Feedback, and Admin. The main content area is titled "TruthLens extension". A purple button at the top right says "Selected: 'Drinking hot water with lemon every hour kills the virus - WHO confirmed'". Below this, a yellow warning box states "▲ LIKELY FAKE (0.91). WHO has not published this claim. Source: WHO Mythbusters, PIB Fact Check." A text box shows the suspicious phrases: "kills the virus", "WHO confirmed". At the bottom, a text box says "Right-click any text and choose 'Check with TruthLens'" with a microphone icon.

TruthLens

[Check News](#) [History](#) [Trending](#) [Feedback](#) [Admin](#)

TruthLens extension

Selected: 'Drinking hot water with lemon every hour kills the virus - WHO confirmed'

▲ LIKELY FAKE (0.91). WHO has not published this claim. Source: WHO Mythbusters, PIB Fact Check.

Suspicious phrases: 'kills the virus', 'WHO confirmed'

Right-click any text and choose 'Check with TruthLens'

TRENDING CLAIMS

https://truthlens.app/

TruthLens

Check NewsHistoryTrendingFeedbackAdmin

Trending claims - last 24 hours

Language: AllVerdict: All

Claim	Lang	Checks	Verdict
ATMs closed for 3 days	ta	1,284	FAKE
New ration card online portal	ta	640	REAL
Free recharge for all users	en	522	FAKE
Cyclone warning for coastal districts	en	498	REAL
Salt shortage rumour	mixed	301	UNCERTAIN

ADMIN DASHBOARD

https://truthlens.app/

TruthLens

Check NewsHistoryTrendingFeedbackAdmin

Admin Dashboard

Checks today
8,240

Fake detected
37%

Feedback pending
112

Model version
v1.4

Checks per day

Day	Checks
Mon	5500
Tue	6000
Wed	5800
Thu	7000
Fri	7500
Sat	6800
Sun	6000

Model performance

Metric	Tamil	English
Accuracy	90.3%	93.0%
Precision	0.89	0.92
Recall	0.90	0.91
F1	0.90	0.92

CHAPTER VIII

CONCLUSION AND FUTURE ENHANCEMENT

8.1 Conclusion

The Fake News and Misinformation Detector successfully classifies Tamil, English and code-mixed messages with an accuracy of 92.1% when model predictions are fused with evidence from trusted fact-check sources. The explainable output with highlighted phrases and source links helps users understand and trust the verdict.

The browser extension and simple web interface make verification a one-click activity. The feedback loop allows the system to keep learning as new rumours appear. The project contributes towards a safer information environment, particularly for regional language users.

8.2 Future Enhancement

- Support for image and video misinformation using OCR and deepfake detection.
- WhatsApp chatbot integration for checking forwarded messages.
- Support for more Indian languages such as Telugu, Malayalam and Hindi.
- Claim-level fact verification using retrieval-augmented generation.
- Early detection of emerging rumours by monitoring trending topics.

CHAPTER IX

BIBLIOGRAPHY

1. Shu, K., Sliva, A., Wang, S., Tang, J., and Liu, H., "Fake News Detection on Social Media: A Data Mining Perspective", ACM SIGKDD Explorations, Vol. 19, No. 1, pp. 22-36, 2017.
2. Wang, W. Y., "'Liar, Liar Pants on Fire': A New Benchmark Dataset for Fake News Detection", Proc. ACL, pp. 422-426, 2017.
3. Vosoughi, S., Roy, D., and Aral, S., "The spread of true and false news online", Science, Vol. 359, No. 6380, pp. 1146-1151, 2018.
4. Devlin, J., Chang, M.-W., Lee, K., and Toutanova, K., "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding", Proc. NAACL-HLT, 2019.
5. Conneau, A. et al., "Unsupervised Cross-lingual Representation Learning at Scale", Proc. ACL, 2020.
6. Ribeiro, M. T., Singh, S., and Guestrin, C., "'Why Should I Trust You?': Explaining the Predictions of Any Classifier", Proc. ACM SIGKDD, 2016.

WEBSITE REFERENCE

1. <https://huggingface.co/docs/transformers>
2. <https://fastapi.tiangolo.com>
3. <https://developers.google.com/fact-check/tools/api>
4. <https://developer.chrome.com/docs/extensions>